PEGASO
Università Telematica

TEMPLATE
PROJECT WORK

Corso di Studio	INFORMATICA PER LE AZIENDE DIGITALI (L-31)
Dimensione dell'elaborato	Minimo 6.000 – Massimo 10.000 parole *(pari a circa Minimo 12 – Massimo 20 pagine)*
Formato del file da caricare in piattaforma	PDF
Nome e Cognome	Francesco Di Maria
Numero di matricola	0312400617
Tema n. (Indicare il numero del tema scelto):	1
Titolo del tema (Indicare il titolo del tema scelto):	La digitalizzazione dell'impresa
Traccia del PW n. (Indicare il numero della traccia scelta):	21
Titolo della traccia (Indicare il titolo della traccia scelta):	Progettare l'analisi dei dati per una startup digitale di servizi on-demand
Titolo dell'elaborato (Attribuire un titolo al proprio elaborato progettuale):	Manolesta: data warehouse a stella, procedura ETL e dashboard interattiva per l'analisi dei dati di una startup di servizi on-demand

PARTE PRIMA – DESCRIZIONE DEL PROCESSO

Utilizzo delle conoscenze e abilità derivate dal percorso di studio
(Descrivere quali conoscenze e abilità apprese durante il percorso di studio sono state utilizzate per la redazione dell'elaborato, facendo eventualmente riferimento agli insegnamenti che hanno contribuito a maturarle):

Il project work mette insieme competenze maturate in quasi tutte le aree del corso di laurea. Più che applicare un singolo insegnamento, ho dovuto far dialogare conoscenze diverse: la modellazione dei dati, la programmazione, la statistica, l'organizzazione aziendale e il diritto. Di seguito riporto gli insegnamenti che hanno contribuito in modo più diretto e il modo in cui ne ho usato i contenuti.

Basi di dati

È l'insegnamento su cui poggia l'intero elaborato. Dal corso ho ripreso il modello relazionale, il concetto di chiave primaria ed esterna, i vincoli di integrità e il linguaggio SQL, sia nella parte di definizione delle tabelle (DDL: CREATE TABLE, CHECK, REFERENCES, CREATE INDEX, CREATE VIEW) sia nella parte di interrogazione (join, raggruppamenti, funzioni di aggregazione, espressioni CASE, sottoquery e Common Table Expression). Il corso mi ha dato anche la consapevolezza della differenza tra un database progettato per registrare le operazioni quotidiane, normalizzato per evitare ridondanze, e un database pensato per l'analisi, dove una certa ridondanza controllata è accettata per rendere le interrogazioni più semplici e veloci. Questa distinzione è il punto di partenza della scelta dello schema a stella.

Programmazione 1 e Programmazione 2

Tutto il prototipo è scritto in Python. Da questi insegnamenti ho ripreso la scomposizione del problema in funzioni con una responsabilità precisa, l'organizzazione del codice in moduli, l'uso di

strutture dati come dizionari e liste, la gestione degli argomenti da riga di comando e, dalla parte di programmazione orientata agli oggetti, l'uso di una piccola classe (Report) per raccogliere in un unico oggetto i controlli di qualità e le righe scartate durante l'ETL.

Algoritmi e strutture dati

Le conoscenze di questo insegnamento sono servite soprattutto nella scrittura del generatore di dati e dell'ETL: scegliere strutture adatte alle ricerche (ad esempio i dizionari usati per tradurre i codici di origine nelle chiavi surrogate, con accesso a tempo costante) ed evitare cicli annidati inutili, preferendo operazioni vettoriali sulle colonne delle tabelle.

Calcolo delle probabilità e statistica

Il generatore produce dati inventati ma realistici, e il realismo dipende dalla scelta delle distribuzioni di probabilità. Ho usato la distribuzione log-normale per i tempi di erogazione e per la "fedeltà" dei clienti (valori sempre positivi, con una coda a destra), la normale per gli importi, una distribuzione di Pareto per la popolarità dei fornitori (pochi partner molto richiesti e molti poco richiesti) ed estrazioni pesate per giorni della settimana, ore del giorno e stagionalità. Dalla statistica descrittiva vengono invece gli indicatori della dashboard: medie, percentuali, distribuzioni per categoria e serie storiche.

Matematica discreta

La logica degli insiemi e delle relazioni è alla base sia del modello relazionale sia dei controlli di integrità: verificare che ogni richiesta faccia riferimento a un cliente e a un fornitore esistenti significa, in pratica, verificare l'appartenenza di un elemento a un insieme.

Ingegneria del software

Da questo insegnamento ho tratto l'impostazione del lavoro: analisi dei requisiti a partire dalla traccia, progettazione prima dell'implementazione, separazione delle responsabilità tra componenti (generazione, caricamento, calcolo, presentazione), test automatici e documentazione. In particolare ho scritto otto test con la libreria pytest che controllano l'intera pipeline e ho curato un file README che permette a chiunque di riprodurre il lavoro.

Tecnologie web

La dashboard è un'applicazione web che gira nel browser. Le nozioni sul modello client-server, sul protocollo HTTP e sull'interazione tra interfaccia e dati mi hanno aiutato a capire come funziona Streamlit (un server Python che invia al browser l'interfaccia e la ricalcola a ogni interazione dell'utente) e a usare uno strumento di automazione del browser per catturare gli screenshot del report.

Programmazione distribuita e cloud computing

Il prototipo funziona su un singolo computer, ma le conoscenze di questo insegnamento sono state utili nella valutazione dei limiti e degli sviluppi futuri: il passaggio da un file SQLite a un data warehouse in cloud, l'aggiornamento automatico e periodico dei dati e la pubblicazione della dashboard per più utenti.

Strategia, organizzazione e marketing; Corporate planning e valore d'impresa

Questi insegnamenti mi hanno aiutato a ragionare come il fondatore della startup, che è l'utente finale dello strumento. Gli indicatori scelti non sono solo numeri tecnici: il tasso di ritorno dei

clienti misura la fidelizzazione, gli incassi per area indicano dove concentrare gli investimenti commerciali, il tempo di erogazione e la quota di richieste annullate misurano la qualità del servizio percepita dal cliente. La lettura dei risultati in chiave di pianificazione (capacità degli operatori nelle ore di punta, stagionalità delle categorie) deriva da questi corsi.

Diritto per le aziende digitali

La traccia chiede dati privi di informazioni personali. Le nozioni sul Regolamento (UE) 2016/679 (GDPR), in particolare i principi di minimizzazione e di protezione dei dati fin dalla progettazione, hanno guidato la scelta di identificare i clienti solo con un codice, una zona, una data e il canale di iscrizione, senza nomi, recapiti o indirizzi.

Fasi di lavoro e relativi tempi di implementazione per la predisposizione dell'elaborato
(Descrivere le attività svolte in corrispondenza di ciascuna fase di redazione dell'elaborato. Indicare il tempo dedicato alla realizzazione di ciascuna fase, le difficoltà incontrate e come sono state superate):

Ho organizzato il lavoro in otto fasi, procedendo per passi successivi e verificando ogni componente prima di passare al successivo. La tabella riassume le fasi e il tempo dedicato a ciascuna; di seguito descrivo le attività svolte e le principali difficoltà incontrate.

Fase	Attività principali	Tempo
1. Analisi della traccia	Lettura della traccia, individuazione dei requisiti, scelta dello scenario aziendale	6 ore
2. Studio e progettazione	Ripasso della teoria del data warehouse, progettazione dello schema a stella, definizione dei KPI	10 ore
3. Generatore di dati	Scrittura e taratura di genera_dati.py, verifica della coerenza dei dati	12 ore
4. Schema SQL ed ETL	Scrittura di schema.sql ed etl.py, controlli di qualità	14 ore
5. Query dei KPI e test	Scrittura di kpi.sql, kpi.py, metriche.py e dei test automatici	8 ore
6. Dashboard	Realizzazione dell'interfaccia Streamlit, grafici e filtri	12 ore
7. Riproducibilità e documentazione	Prova da zero, README, script di avvio, screenshot	6 ore
8. Stesura del report	Redazione, revisione e impaginazione	22 ore
Totale		**90 ore**

Fase 1 – Analisi della traccia

Ho iniziato scomponendo la traccia in requisiti verificabili: volumi dei dati (300–600 record per tipologia), struttura del modello (tabella principale delle richieste e quattro tabelle di supporto), funzioni dell'ETL (lettura, pulizia, caricamento in SQLite), indicatori e grafici obbligatori, filtri della dashboard e contenuti del report. Questo elenco è diventato la lista di controllo con cui, alla fine, ho verificato di aver coperto tutte le richieste. Ho poi scelto uno scenario concreto per rendere i dati più credibili: una startup fittizia, che ho chiamato Manolesta, che tramite app organizza interventi a domicilio (idraulica, elettricità, pulizie, climatizzazione e altri) in sette zone della Sicilia, appoggiandosi a ditte partner e ai loro operatori.

Fase 2 – Studio e progettazione

Ho ripreso i concetti di data warehouse, fatti, dimensioni e granularità, e ho disegnato lo schema a stella prima di scrivere codice. La decisione più importante è stata la grana della tabella dei fatti: una riga per ogni richiesta. In questa fase ho anche scritto le definizioni precise dei KPI. La difficoltà principale è stata proprio questa: espressioni come "tempo medio di erogazione" o "clienti che

tornano" sembrano chiare, ma vanno rese univoche (le richieste annullate entrano nel calcolo? il ritorno si misura sull'intero storico o sul periodo selezionato?). Ho risolto scrivendo una definizione formale per ogni indicatore, riportata nel README e nei commenti del codice SQL.

Fase 3 – Generatore di dati

È stata la fase più delicata, perché dati incoerenti avrebbero reso inutili tutte le analisi successive. Ho imposto alcuni vincoli: un cliente può fare richieste solo dopo la data di iscrizione; fornitore e operatore devono essere della stessa zona del cliente e della categoria richiesta; ogni combinazione di zona e categoria deve avere almeno un fornitore e un operatore. Una prima difficoltà è emersa controllando i risultati: nella prima versione la fedeltà dei clienti seguiva una distribuzione di Pareto troppo concentrata, e un solo cliente risultava autore di oltre cento richieste, un valore irrealistico. Ho sostituito la distribuzione con una log-normale, ottenendo un massimo di 15 richieste per cliente e circa metà dei clienti attivi che ritornano. Una seconda difficoltà riguardava il rumore statistico: con solo 600 richieste in un anno, la serie mensile oscillava molto da un seme casuale all'altro (in una prova febbraio scendeva a 20 richieste, la metà di gennaio). Ho confrontato otto semi diversi e ho scelto come valore predefinito quello che restituisce una serie leggibile, documentando la scelta e lasciando il seme modificabile da riga di comando.

Fase 4 – Schema SQL ed ETL

Ho scritto lo script di creazione del database e la procedura di caricamento. Per mettere davvero alla prova la pulizia ho aggiunto al generatore una funzione che "sporca" di proposito circa il 3% delle righe con i difetti tipici dei dati reali: maiuscole e spazi incoerenti, importi scritti con la virgola, date in due formati, stati scritti in modi diversi, duplicati, operatori mancanti, importi negativi e codici di fornitori inesistenti. La difficoltà è stata decidere, per ogni difetto, se correggere o scartare. Ho adottato una regola semplice: si corregge quando il valore corretto è ricavabile senza ambiguità (una zona scritta in maiuscolo, una data in altro formato), si scarta quando il dato è sbagliato e non ricostruibile (un importo negativo), e si conserva la riga collegandola a un membro "Sconosciuto" quando manca un'informazione non essenziale (l'operatore). Le righe scartate sono salvate in un file con il motivo, così nulla si perde in silenzio.

Fase 5 – Query dei KPI e test

Ho scritto le query SQL degli indicatori e un modulo Python che calcola gli stessi KPI per la dashboard. Avere due implementazioni della stessa definizione espone al rischio che divergano; per questo ho scritto un test che le confronta sull'intero dataset. Gli altri test verificano i volumi, la riproducibilità del generatore, l'assenza di dati personali, l'integrità del warehouse e la corretta registrazione degli scarti.

Fase 6 – Dashboard

Ho realizzato la dashboard con Streamlit e Plotly. Le difficoltà sono state soprattutto di leggibilità: la libreria grafica mostrava i mesi in inglese ("May 2025") e tagliava le etichette dei valori nel grafico dei fornitori. Ho risolto costruendo le etichette dei mesi in italiano e allargando l'asse. Ho inoltre formattato numeri e importi secondo le convenzioni italiane (punto per le migliaia, virgola per i decimali).

Fase 7 – Riproducibilità e documentazione

Ho cancellato tutti i dati generati e ho rieseguito l'intera pipeline da zero a partire dall'archivio del progetto, verificando che i file prodotti fossero identici byte per byte a quelli originali e che tutti i

test fossero superati. Per gli screenshot ho usato uno script di automazione del browser; la difficoltà è stata che Streamlit scorre il contenuto all'interno di un contenitore e la cattura "a pagina intera" ne mostrava solo una parte. Ho risolto allargando la finestra virtuale all'altezza effettiva del contenuto prima della cattura.

Fase 8 – Stesura del report

Infine ho redatto questo elaborato seguendo il template, riportando i risultati reali ottenuti dal prototipo e verificando che ogni numero citato corrispondesse all'output delle query.

Risorse e strumenti impiegati
(Descrivere quali risorse - bibliografia, banche dati, ecc. - e strumenti - software, modelli teorici, ecc. - sono stati individuati ed utilizzati per la redazione dell'elaborato. Descrivere, inoltre, i motivi che hanno orientato la scelta delle risorse e degli strumenti, la modalità di individuazione e reperimento delle risorse e degli strumenti, le eventuali difficoltà affrontate nell'individuazione e nell'utilizzo di risorse e strumenti ed il modo in cui sono state superate):

Strumenti software

La scelta degli strumenti è stata guidata da tre criteri dettati dalla traccia: semplicità, riproducibilità in pochi minuti e gratuità. Ho preferito strumenti open source, installabili con un solo comando e funzionanti su Windows, macOS e Linux.

Strumento	Uso nel progetto	Motivo della scelta
Python 3	Linguaggio di tutto il prototipo	Diffuso nell'analisi dei dati, studiato nel corso, ricco di librerie
pandas e NumPy	Generazione, pulizia e trasformazione dei dati	Standard di fatto per la manipolazione di tabelle in Python
SQLite	Database del data warehouse	Richiesto dalla traccia; un solo file, nessun server da installare
SQL	Definizione dello schema e calcolo dei KPI	Linguaggio standard per interrogare i database relazionali
Streamlit	Dashboard interattiva	Indicato dalla traccia; permette un'interfaccia web scrivendo solo Python
Plotly	Grafici della dashboard	Grafici interattivi (zoom, valori al passaggio del mouse)
pytest	Test automatici	Libreria di test semplice e molto diffusa
Git e GitHub	Versionamento del codice e pubblicazione del progetto	Standard di fatto per la gestione e la condivisione del codice sorgente
Graphviz	Diagrammi dello schema a stella e del flusso dei dati	Diagrammi generati da testo, quindi versionabili insieme al codice
Playwright	Screenshot della dashboard (facoltativo)	Automazione del browser riproducibile
Visual Studio Code	Ambiente di sviluppo	Editor gratuito con supporto a Python e SQL

Tra gli strumenti di visualizzazione suggeriti dalla traccia (Streamlit, Power BI, Tableau Public) ho scelto Streamlit perché è l'unico che consente di tenere tutto il progetto in codice: la dashboard si avvia con un comando, è versionabile come il resto del progetto e non richiede licenze, account o sistemi operativi specifici. Power BI Desktop, ad esempio, funziona solo su Windows, e Tableau Public pubblica i dati online, cosa non necessaria per un prototipo.

Modelli teorici

Il riferimento teorico principale è la modellazione dimensionale di Ralph Kimball: progettazione per processo di business, scelta della grana, individuazione di dimensioni e fatti, uso di chiavi surrogate e di membri "sconosciuti". Ho considerato anche l'approccio di Bill Inmon, che prevede un data warehouse aziendale normalizzato da cui derivare i data mart, ma per un prototipo con un solo processo di business (la gestione delle richieste) l'approccio dimensionale è più adatto. Per la dashboard ho seguito i principi di Stephen Few sulla progettazione di cruscotti informativi: pochi indicatori in evidenza in alto, un colore principale per le serie singole, nessun elemento decorativo che non trasmetta informazione.

Risorse bibliografiche e documentazione

- R. Kimball, M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3ª ed., Wiley, 2013.
- W. H. Inmon, *Building the Data Warehouse*, 4ª ed., Wiley, 2005.
- M. Golfarelli, S. Rizzi, *Data Warehouse. Teoria e pratica della progettazione*, 2ª ed., McGraw-Hill, 2006.
- P. Atzeni, S. Ceri, S. Paraboschi, R. Torlone, *Basi di dati*, McGraw-Hill.
- S. Few, *Information Dashboard Design*, 2ª ed., Analytics Press, 2013.
- Regolamento (UE) 2016/679 (GDPR), artt. 5 e 25.
- Documentazione ufficiale di Python, pandas, SQLite, Streamlit e Plotly.
- Materiali didattici degli insegnamenti del corso di laurea citati nella sezione precedente.

Modalità di reperimento e difficoltà

I testi di riferimento sono classici della materia, citati anche nei materiali dei corsi; la documentazione degli strumenti è disponibile gratuitamente sui siti ufficiali. Non avendo accesso a dati reali di un'azienda, la "banca dati" del progetto è stata prodotta dal generatore stesso: questa è insieme una richiesta della traccia e la principale difficoltà affrontata, perché ho dovuto stabilire da solo parametri plausibili (prezzi medi, tempi di intervento, stagionalità, tasso di annullamento) ragionando su come funziona un servizio di questo tipo. Li ho raccolti tutti in un unico file di configurazione, così da poterli rivedere e modificare facilmente. Una seconda difficoltà ha riguardato la compatibilità tra versioni delle librerie: per evitarla ho indicato nel file requirements.txt le versioni minime necessarie.

Durante il lavoro ho utilizzato anche un assistente di intelligenza artificiale generativa come supporto per la revisione del codice e la verifica di alcune scelte tecniche; tutte le scelte progettuali e i risultati riportati sono stati controllati eseguendo il prototipo.

PARTE SECONDA – PREDISPOSIZIONE DELL'ELABORATO

Obiettivi del progetto
(Descrivere gli obiettivi raggiunti dall'elaborato, indicando in che modo esso risponde a quanto richiesto dalla traccia):

L'obiettivo generale del project work è dare al fondatore di una startup di servizi on-demand uno strumento che riunisca i dati sparsi dei diversi sistemi aziendali in un unico archivio organizzato e li renda leggibili attraverso indicatori e grafici. L'elaborato raggiunge l'obiettivo con un prototipo funzionante, composto da quattro elementi: un generatore di dati sintetici, un data warehouse a stella in SQLite, una procedura ETL e una dashboard interattiva. La tabella seguente mette in corrispondenza ogni obiettivo della traccia con il modo in cui è stato soddisfatto.

Richiesta della traccia	Come è stata soddisfatta
Dati inventati ma realistici, 300–600 record per tipologia	500 clienti, 300 fornitori, 350 operatori e 600 richieste, con stagionalità, fasce orarie di picco, crescita mensile, ritardi e annullamenti
Esportazione in CSV, un file per tipologia	Quattro file in data/raw: clienti, fornitori, operatori, richieste
Data warehouse "a stella" con tabella Richieste e dimensioni Cliente, Fornitore, Operatore, Tempo	Tabella dei fatti fact_richieste e dimensioni dim_cliente, dim_fornitore, dim_operatore, dim_tempo in un file SQLite
ETL: lettura dei CSV, pulizia, caricamento	etl.py corregge formati e duplicati, scarta e documenta le righe non valide, carica il warehouse e ne verifica l'integrità
Query SQL dei KPI principali	kpi.sql: numero di richieste, incassi, tempo medio, tasso di ritorno, più analisi per mese, zona, categoria, fornitore e fascia oraria
Dashboard con KPI, grafici e filtri	Dashboard Streamlit con 8 indicatori, andamento nel tempo, primi 5 fornitori, distribuzione per zona e per categoria, mappa giorno × ora; filtri per periodo, zona e categoria
Script ordinati, README, dataset condivisibile	Codice organizzato in cartelle, README con istruzioni, dataset e file SQLite pubblicati nel repository GitHub del progetto (link nella sezione sugli aspetti progettuali)
Dati senza informazioni personali	I clienti sono identificati solo da codice, zona, data e canale di iscrizione
Prototipo riproducibile in pochi minuti	Un solo comando (python esegui_tutto.py) esegue tutto e apre la dashboard; seme casuale fisso

Oltre ai requisiti minimi, il prototipo raggiunge alcuni obiettivi aggiuntivi che ne aumentano l'affidabilità:

- **controllo della qualità dei dati**: ogni esecuzione dell'ETL produce un report con il numero di righe lette, corrette e scartate, e un file con le righe scartate e il motivo dello scarto;
- **coerenza degli indicatori**: le definizioni dei KPI sono implementate in SQL e in Python e un test automatico verifica che diano lo stesso risultato;
- **confronto con il periodo precedente**: quando si seleziona un intervallo di date, la dashboard mostra la variazione di ogni indicatore rispetto al periodo precedente di pari durata;
- **test automatici**: otto test controllano la pipeline dall'inizio alla fine.

Sul piano didattico, l'obiettivo era mostrare la padronanza dell'intero ciclo di vita del dato in un contesto aziendale: dalla sua origine nei sistemi operativi, attraverso la pulizia e l'integrazione, fino alla presentazione per le decisioni. Il focus richiesto dalla traccia (chiarezza del modello dei dati, correttezza delle trasformazioni, leggibilità della dashboard) è stato rispettato privilegiando soluzioni semplici e ben documentate rispetto a soluzioni più sofisticate ma meno comprensibili.

Contestualizzazione
(Descrivere il contesto teorico e quello applicativo dell'elaborato realizzato):

Contesto teorico

Sistemi operazionali e sistemi analitici

Un'azienda registra le proprie attività in sistemi detti operazionali (OLTP, On-Line Transaction

Processing): l'app con cui il cliente prenota, il gestionale degli operatori, l'anagrafica dei fornitori. Questi sistemi sono progettati per eseguire molte piccole operazioni (inserire una richiesta, aggiornare uno stato) in modo rapido e senza errori, e per questo i loro database sono normalizzati: ogni informazione è memorizzata una sola volta. Questa organizzazione è ideale per registrare i dati, ma poco adatta ad analizzarli: per rispondere a una domanda come "quanto abbiamo incassato a Catania nel terzo trimestre per categoria di servizio?" bisogna unire molte tabelle, spesso su sistemi diversi, e le interrogazioni pesanti rallentano il lavoro quotidiano.

I sistemi analitici (OLAP, On-Line Analytical Processing) nascono per risolvere questo problema. Il loro cuore è il data warehouse, che Inmon definisce come una raccolta di dati orientata ai soggetti, integrata, non volatile e variabile nel tempo, a supporto delle decisioni. Integrata significa che dati provenienti da fonti diverse vengono resi coerenti (stesse codifiche, stessi formati); non volatile significa che i dati vengono caricati e consultati, non modificati dalle operazioni quotidiane; variabile nel tempo significa che conserva la storia.

La modellazione dimensionale e lo schema a stella

Kimball propone di organizzare i dati analitici secondo il modello dimensionale, che distingue due tipi di tabelle:

- le **tabelle dei fatti**, che registrano gli eventi misurabili del processo di business (nel nostro caso le richieste di servizio) e contengono le misure numeriche (importo, tempo di erogazione);
- le **tabelle delle dimensioni**, che descrivono il contesto degli eventi: chi (cliente, operatore), con chi (fornitore), quando (tempo), dove (zona).

Quando una tabella dei fatti è collegata direttamente a tabelle di dimensione non normalizzate, il disegno assume la forma di una stella, da cui il nome. Lo schema a stella ha tre vantaggi principali: è facile da capire anche per chi non è tecnico, perché rispecchia il modo in cui un manager formula le domande ("le richieste per zona e per mese"); le interrogazioni sono semplici, con un solo livello di join tra fatti e dimensioni; le prestazioni sono buone, perché i database sono ottimizzati per questo tipo di accesso. L'alternativa principale, lo schema a fiocco di neve (snowflake), normalizza ulteriormente le dimensioni (ad esempio separando le categorie dei fornitori in una tabella a parte): risparmia un po' di spazio ma rende le query più complesse, senza vantaggi apprezzabili per volumi piccoli come quelli del prototipo.

Il passo più importante della progettazione è la scelta della grana, cioè di che cosa rappresenta una riga della tabella dei fatti. Kimball raccomanda la grana più fine disponibile, perché permette qualsiasi aggregazione successiva. Altri concetti utilizzati nel progetto sono le chiavi surrogate (identificativi interni al warehouse, indipendenti dai codici dei sistemi di origine), la dimensione tempo come calendario completo e le dimensioni che cambiano lentamente nel tempo (Slowly Changing Dimensions), di cui si parla nei limiti.

Il processo ETL

I dati arrivano al warehouse attraverso un processo in tre passi: Estrazione dalle fonti, Trasformazione (pulizia, uniformazione delle codifiche, controlli di qualità, calcolo di campi derivati) e Caricamento nelle tabelle di destinazione. Nella pratica aziendale la trasformazione è la parte più onerosa, perché i dati reali contengono errori, duplicati e formati incoerenti. Un ETL ben fatto non si limita a correggere: rende tracciabile ciò che è stato scartato e perché.

KPI e dashboard

Un indicatore chiave di prestazione (KPI) è una misura sintetica collegata a un obiettivo aziendale.

Perché sia utile deve avere una definizione univoca e condivisa. Una dashboard è una rappresentazione visiva dei KPI più importanti, disposta in modo da essere letta con un solo colpo d'occhio: secondo Few, la dashboard deve permettere di capire in pochi secondi se le cose vanno bene o se serve un intervento.

Contesto applicativo

Il mercato dei servizi on-demand

I servizi on-demand (su richiesta) sono piattaforme digitali che mettono in contatto, in tempo reale, chi ha bisogno di un servizio con chi lo può erogare: trasporto, consegne, servizi per la casa. Il modello si basa su tre attori: il cliente, che fa la richiesta tramite app; il fornitore partner, che eroga il servizio; la piattaforma, che gestisce l'incontro, il pagamento e la qualità. Per una startup di questo tipo i dati sono un patrimonio fondamentale: la velocità di risposta, la capacità di trattenere i clienti e la scelta delle aree su cui investire determinano la sopravvivenza dell'impresa.

Lo scenario: Manolesta

Per rendere concreto il prototipo ho immaginato una startup, Manolesta, che organizza interventi a domicilio in sette zone della Sicilia (Palermo, Catania, Messina, Siracusa, Ragusa, Trapani, Agrigento) e in sette categorie di servizio: idraulica, elettricità, pulizie, climatizzazione, riparazione di elettrodomestici, giardinaggio e piccoli traslochi. Il cliente apre l'app, sceglie il servizio e invia la richiesta; la piattaforma la assegna a una ditta partner della stessa zona, che invia un proprio operatore. Al termine la richiesta viene chiusa con l'importo pagato. Una richiesta può essere annullata prima dell'intervento, oppure chiusa in ritardo se il tempo di erogazione supera il tempo massimo previsto per la sua categoria.

Attività operative e fonti di dati simulate

La traccia descrive quattro sistemi aziendali che producono dati separati. Nel prototipo ciascuno è simulato da un file CSV:

Sistema aziendale	File simulato	Contenuto
App utilizzata dai clienti	clienti.csv	codice cliente, zona, data e canale di iscrizione (app Android, app iOS, sito web)
Archivio dei fornitori partner	fornitori.csv	codice, nome della ditta, categoria di servizio, zona
Sistema di gestione degli operatori	operatori.csv	codice, zona, tipo di attività, anni di esperienza
Piattaforma che gestisce le richieste	richieste.csv	codice richiesta, cliente, fornitore, operatore, data e ora, importo, tempo di erogazione, stato

La situazione di partenza descritta dalla traccia, con dati sparsi in file e database diversi, è stata riprodotta anche nei suoi aspetti problematici: i file contengono le piccole incoerenze tipiche di sistemi sviluppati separatamente (codifiche diverse degli stati, formati di data diversi, importi scritti all'italiana), che l'ETL deve riconciliare.

Descrizione dei principali aspetti progettuali
(Sviluppare l'elaborato richiesto dalla traccia prescelta):

1. Architettura del prototipo

Il prototipo è organizzato come una catena di passi indipendenti, ciascuno con un compito preciso e un risultato salvato su file. Questa struttura permette di eseguire e verificare ogni passo separatamente e di sostituirne uno senza toccare gli altri: ad esempio, in futuro i CSV sintetici potrebbero essere sostituiti da estrazioni reali senza modificare il warehouse e la dashboard.

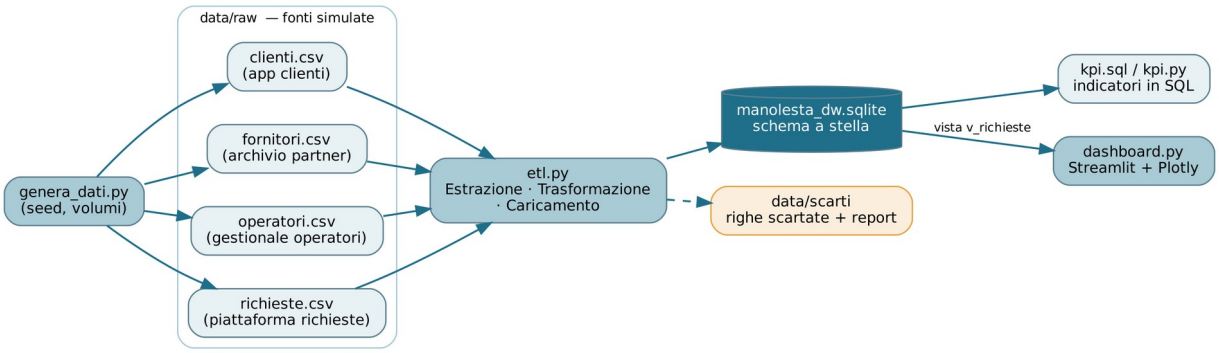


Figura 1 – Flusso dei dati: dalla generazione dei CSV alla dashboard

Materiali del progetto. Il codice sorgente (script Python e SQL), il dataset sintetico in formato CSV, il file SQLite del data warehouse, gli screenshot e il file README con le istruzioni sono pubblicati in un repository GitHub pubblico, consultabile online senza registrazione e scaricabile per intero come archivio ZIP (pulsante "Code", voce "Download ZIP"): https://github.com/ChiccoHD/pw21-manolesta

Il progetto è organizzato nelle seguenti cartelle: src (codice Python), sql (schema e query), app (dashboard), tests (test automatici), data (CSV, warehouse e scarti), docs (diagrammi e screenshot). L'intera pipeline si esegue con un solo comando:

```
pip install -r requirements.txt
python esegui_tutto.py
```

2. Generatore dei dati sintetici

Il modulo genera_dati.py produce i quattro CSV. Tutti i parametri (zone e loro peso, categorie con prezzo medio, tempo medio e tempo massimo previsto, stagionalità, pesi dei giorni e delle ore) sono raccolti nel file config.py, così che lo scenario possa essere modificato senza toccare la logica.

Anagrafiche

I clienti ricevono una zona estratta in proporzione al peso della zona (Palermo e Catania sono le più grandi) e una data di iscrizione compresa tra il lancio del servizio (settembre 2024) e la fine del 2025, con densità crescente nel tempo per simulare la crescita della base clienti. I fornitori e gli operatori sono distribuiti tra zone e categorie garantendo che ogni combinazione abbia almeno un fornitore e un operatore; i nomi delle ditte sono combinazioni casuali di parole (ad esempio "Futura Clima Point"), quindi non corrispondono ad aziende reali.

Richieste

Per ogni richiesta il generatore sceglie la categoria, poi il giorno, combinando tre fattori: una crescita mensile del 6% (la startup è in espansione), il giorno della settimana (lunedì più carico, domenica molto meno) e la stagionalità della categoria (la climatizzazione ha un picco a luglio e agosto, il giardinaggio in primavera). L'ora è estratta con due picchi, a metà mattina e a fine pomeriggio. Il cliente è scelto tra quelli già iscritti, pesato in base a una "fedeltà" individuale con distribuzione log-normale; il fornitore tra quelli della stessa zona e categoria, pesato per popolarità; l'operatore tra quelli della zona con la competenza richiesta.

L'esito della richiesta segue regole semplici ma realistiche:

- la probabilità di annullamento è più alta nelle ore di punta (11% contro 7%), quando le attese sono più lunghe;
- il tempo di erogazione è estratto attorno al tempo medio della categoria, aumenta del 15% nelle ore di punta, diminuisce con l'esperienza dell'operatore e, nell'8% dei casi, subisce un imprevisto che lo allunga fino a più del doppio;
- la richiesta è "in ritardo" se il tempo supera il massimo previsto per la categoria;
- l'importo è estratto attorno al prezzo medio della categoria, con una maggiorazione del 20% per interventi serali e nei fine settimana.

Difetti introdotti di proposito

Per verificare la pulizia, il generatore altera circa il 3% delle righe con i difetti descritti nella sezione sulle fasi di lavoro. Con l'opzione --pulito i dati vengono esportati senza difetti. Il generatore usa un seme casuale fisso: con gli stessi parametri produce sempre gli stessi file.

Parametro	Valore predefinito	Opzione
Seme casuale	2	--seed
Clienti / fornitori / operatori / richieste	500 / 300 / 350 / 600	--clienti --fornitori --operatori --richieste
Periodo delle richieste	01/01/2025 – 31/12/2025	config.py
Quota di righe con difetti	3%	--pulito per disattivarli

3. Il data warehouse a stella

Grana e struttura

La tabella dei fatti fact_richieste ha la grana più fine disponibile: una riga per ogni richiesta di servizio. È collegata a quattro dimensioni, come richiesto dalla traccia.

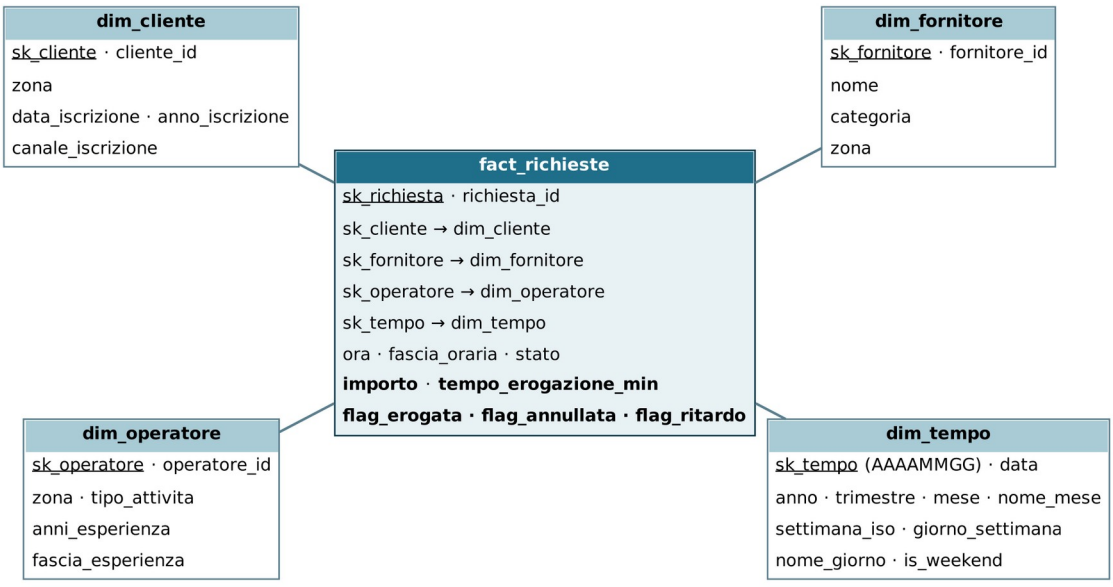


Figura 2 – Schema a stella del data warehouse

Le misure della tabella dei fatti sono l'importo e il tempo di erogazione; a queste si aggiungono tre indicatori 0/1 (richiesta erogata, annullata, in ritardo) che rendono i KPI calcolabili con semplici somme e medie. Ad esempio, la percentuale di richieste annullate è la media di flag_annullata. L'ora

e la fascia oraria sono memorizzate direttamente nella tabella dei fatti: una dimensione a parte per un dato con 24 valori avrebbe complicato lo schema senza vantaggi. Il codice della richiesta è conservato come dimensione degenere, cioè un attributo senza tabella propria, utile per risalire al dato di origine.

```
CREATE TABLE fact_richieste (
    sk_richiesta          INTEGER PRIMARY KEY,
    richiesta_id          TEXT    NOT NULL UNIQUE,
    sk_cliente            INTEGER NOT NULL REFERENCES dim_cliente(sk_cliente),
    sk_fornitore          INTEGER NOT NULL REFERENCES dim_fornitore(sk_fornitore),
    sk_operatore          INTEGER NOT NULL REFERENCES dim_operatore(sk_operatore),
    sk_tempo              INTEGER NOT NULL REFERENCES dim_tempo(sk_tempo),
    ora                   INTEGER NOT NULL CHECK (ora BETWEEN 0 AND 23),
    fascia_oraria         TEXT    NOT NULL,
    stato                 TEXT    NOT NULL CHECK (stato IN
('completata','in_ritardo','annullata')),
    importo               REAL    NOT NULL CHECK (importo >= 0),
    tempo_erogazione_min  INTEGER,
    flag_erogata INTEGER NOT NULL, flag_annullata INTEGER NOT NULL, flag_ritardo
INTEGER NOT NULL
);
```

Perché lo schema a stella

Ho scelto l'organizzazione a stella, oltre che perché richiesta dalla traccia, per tre ragioni legate agli obiettivi del progetto. La prima è la chiarezza: lo schema si spiega con un disegno e le sue tabelle corrispondono ai concetti che il fondatore usa ogni giorno. La seconda è la semplicità delle interrogazioni: ogni KPI si ottiene con un join tra fatti e dimensioni seguito da un raggruppamento. La terza è l'estendibilità: aggiungere una nuova dimensione (ad esempio il canale di pagamento) o una nuova misura (ad esempio la valutazione del cliente) non richiede di riprogettare le tabelle esistenti.

Scelte di dettaglio

- **Chiavi surrogate**: ogni dimensione ha una chiave numerica interna (sk_cliente, sk_fornitore, …) distinta dal codice di origine. Il warehouse non dipende così dai codici dei sistemi sorgente, e in futuro la stessa entità potrà avere più versioni storiche.
- **Membro "Sconosciuto"**: ogni dimensione contiene una riga con chiave 0 che rappresenta un valore mancante. Le richieste senza operatore registrato restano nelle analisi invece di essere perse.
- **Dimensione tempo completa**: dim_tempo contiene tutti i 365 giorni del 2025, anche quelli senza richieste, con gli attributi utili all'analisi (mese, trimestre, settimana, giorno della settimana, fine settimana). La chiave è nel formato AAAAMMGG, leggibile e ordinabile.
- **Zona di analisi**: la zona usata nei filtri e nei grafici è quella del cliente, cioè il luogo in cui il servizio viene erogato.
- **Vincoli e indici**: chiavi esterne, vincoli CHECK e indici sulle chiavi della tabella dei fatti garantiscono l'integrità e la velocità delle interrogazioni.
- **Vista per la dashboard**: la vista v_richieste unisce fatti e dimensioni in una tabella "piatta" che la dashboard legge con una sola query.

4. La procedura ETL

Il modulo etl.py svolge le tre fasi in funzioni separate.

Estrazione. I quattro CSV vengono letti come testo, senza conversioni automatiche: in questo modo nessun valore viene interpretato in modo errato prima dei controlli (ad esempio "89,50" non diventa un valore mancante).

Trasformazione. Le regole di pulizia sono riassunte nella tabella seguente.

Problema nei dati di origine	Trattamento
Maiuscole e spazi incoerenti (" PALERMO", "catania ")	Ricondotti al valore canonico dell'elenco delle zone e delle categorie
Importi con la virgola decimale ("89,50")	Convertiti in numero
Date in due formati (2025-03-14 10:30 e 14/03/2025 10:30)	Entrambi riconosciuti
Stati scritti in modi diversi ("Completata", "annullato", "ritardo")	Normalizzati in completata, in_ritardo, annullata
Righe duplicate	Rimosse
Importi negativi, clienti o fornitori inesistenti	Scartati e salvati con il motivo
Operatore mancante	Collegato al membro "Sconosciuto"
Richieste annullate con importo o tempo valorizzato	Importo azzerato e tempo rimosso

La trasformazione calcola anche i campi derivati (fascia oraria, fascia di esperienza dell'operatore, indicatori 0/1). Le funzioni di pulizia sono generiche e riutilizzabili; ad esempio, quella che uniforma i valori testuali è la seguente:

```
def uniforma(s, valori_ammessi):
    """Riconduce varianti di maiuscole/spazi al valore canonico; NaN se
sconosciuto."""
    mappa = {v.lower(): v for v in valori_ammessi}
    return pulisci_testo(s).str.lower().map(mappa)
```

Caricamento. Lo schema viene ricreato da schema.sql, le dimensioni vengono caricate con le chiavi surrogate e i codici di origine della tabella dei fatti vengono sostituiti con le chiavi corrispondenti. Al termine l'ETL esegue due controlli automatici: verifica che non esistano chiavi esterne non valide e che il totale degli incassi caricati coincida con quello dei dati validi; se un controllo fallisce, la procedura si interrompe con un errore.

Sull'esecuzione con i parametri predefiniti l'ETL ha letto 606 richieste, rimosso 6 duplicati, scartato 3 righe con importo negativo e 2 con fornitore inesistente, collegato 9 richieste al membro "Sconosciuto" e caricato 595 richieste, per un incasso complessivo di 55.471 euro coerente con i dati di origine. Tutte le anagrafiche sono state caricate senza scarti dopo l'uniformazione di zone e categorie.

5. Le interrogazioni SQL dei KPI

Il file kpi.sql contiene le query degli indicatori, eseguibili con il comando python -m src.kpi. Le definizioni adottate sono le seguenti:

- **richieste totali**: tutte le richieste valide, comprese le annullate;
- **incassi**: somma degli importi delle richieste erogate (completate o in ritardo);
- **tempo medio di erogazione**: media dei minuti tra la richiesta e la chiusura dell'intervento, calcolata sulle sole richieste erogate;
- **clienti attivi**: clienti distinti con almeno una richiesta erogata nel periodo;
- **tasso di ritorno**: quota dei clienti attivi che hanno almeno due richieste erogate nel periodo.

Il tasso di ritorno è l'indicatore più interessante dal punto di vista SQL, perché richiede due livelli di aggregazione: prima si contano le richieste per cliente, poi si calcola la quota di clienti con almeno due richieste.

```
WITH per_cliente AS (
    SELECT sk_cliente, COUNT(*) AS n_richieste
```

```
    FROM fact_richieste
    WHERE flag_erogata = 1
    GROUP BY sk_cliente
)
SELECT COUNT(*)                                          AS clienti_attivi,
       SUM(n_richieste >= 2)                             AS clienti_di_ritorno,
       ROUND(100.0 * SUM(n_richieste >= 2) / COUNT(*), 1) AS tasso_ritorno_perc
FROM per_cliente;
```

Oltre ai quattro KPI richiesti, il file contiene query di analisi per mese, zona, categoria, fornitore e fascia oraria. I risultati sull'intero 2025 sono i seguenti.

Indicatore	Valore 2025
Richieste totali	595
Richieste erogate	549
Incassi	55.471 €
Tempo medio di erogazione	133 minuti
Clienti attivi	223
Clienti che tornano (2+ richieste)	110 (49,3%)
Richieste annullate	7,7%
Richieste erogate in ritardo	8,4%
Scontrino medio	101 €

6. La dashboard

La dashboard è realizzata con Streamlit e si avvia con il comando streamlit run app/dashboard.py. Legge la vista v_richieste una sola volta (il risultato viene tenuto in memoria) e ricalcola indicatori e grafici a ogni modifica dei filtri.

L'interfaccia è organizzata dall'alto verso il basso, dal dato più sintetico al più dettagliato:

- **filtri** nella barra laterale: periodo, zona, categoria di servizio e granularità dell'andamento (settimana o mese);
- **otto indicatori** in evidenza: i quattro richiesti dalla traccia (richieste totali, incassi, tempo medio di erogazione, clienti attivi) più il tasso di ritorno, la quota di annullate, la quota di richieste in ritardo e lo scontrino medio; quando il periodo selezionato non copre l'intero anno, ogni indicatore mostra la variazione rispetto al periodo precedente di pari durata;
- **grafici**: andamento delle richieste e delle annullate nel tempo, classifica dei primi cinque fornitori per incassi, richieste per zona, richieste per categoria di servizio e una mappa di calore giorno × ora che mostra quando arrivano le richieste;
- **dettaglio**: una tabella riassuntiva per zona e l'elenco delle richieste filtrate, scaricabile in CSV.

Per la leggibilità ho seguito poche regole: un solo colore per le serie singole e un secondo colore solo per le annullate; valori scritti direttamente sulle barre; numeri e date in formato italiano; una breve spiegazione della definizione di ogni KPI visibile al passaggio del mouse sul simbolo "?".

7. Test e riproducibilità

Otto test automatici, eseguibili con python -m pytest, verificano: che i volumi rispettino la traccia; che il generatore sia riproducibile (due esecuzioni con lo stesso seme producono file identici); che i clienti non contengano dati personali; che nel warehouse non ci siano chiavi esterne non valide, importi negativi, duplicati o codifiche non canoniche; che la dimensione tempo sia completa; che gli scarti siano registrati; che i KPI calcolati in SQL e in Python coincidano; che, con dati privi di

difetti, l'ETL non scarti nulla. I test lavorano in una cartella temporanea e non modificano i dati del progetto. Come prova finale ho estratto il progetto in una cartella vuota e rieseguito tutto da zero: la pipeline completa richiede pochi secondi e produce file identici a quelli consegnati.

8. Risultati

Le figure seguenti mostrano la dashboard con i dati dell'intero 2025 e senza filtri.



Figura 3 – Indicatori principali (01/01/2025 – 31/12/2025, tutte le zone e categorie)

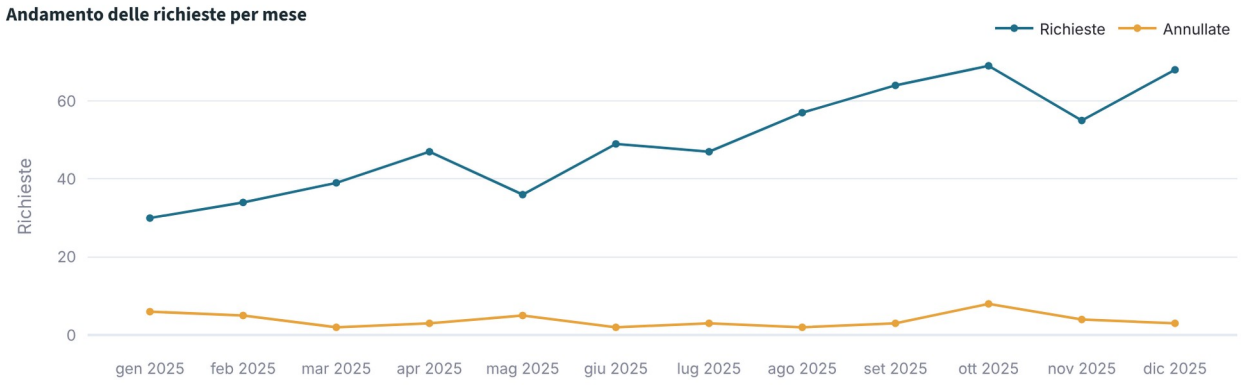


Figura 4 – Andamento mensile delle richieste e delle richieste annullate

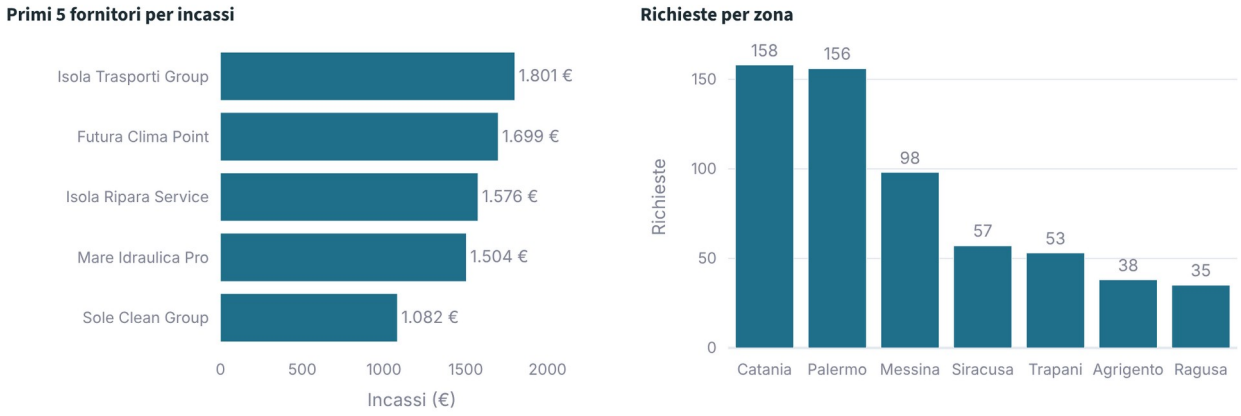


Figura 5 – Primi cinque fornitori per incassi e richieste per zona

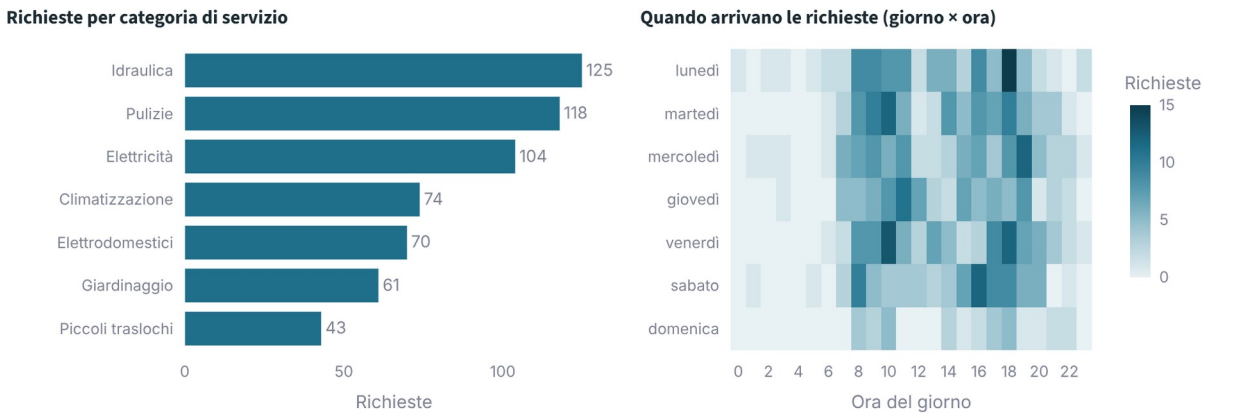


Figura 6 – Richieste per categoria di servizio e distribuzione per giorno e ora

La Figura 7 mostra l'effetto dei filtri: selezionando le zone di Palermo e Catania gli indicatori si aggiornano (314 richieste e 28.852 euro di incassi, cioè il 52% del totale).



Figura 7 – Dashboard filtrata sulle zone di Palermo e Catania

Osservazioni principali

Prima di commentare i risultati è necessaria una precisazione di metodo: i dati sono sintetici, quindi gli andamenti osservati riflettono le ipotesi inserite nel generatore. Il valore delle osservazioni non sta nella scoperta di fenomeni reali, ma nella dimostrazione che il prototipo è in grado di farli emergere in modo chiaro, come farebbe con dati veri.

1. La crescita è sostenuta e si concentra in due zone. Le richieste passano da 103 nel primo trimestre a 192 nel quarto (+86%) e gli incassi da circa 9.000 a circa 17.200 euro (+91%); il secondo semestre vale il 63% degli incassi dell'anno. Palermo e Catania generano da sole il 52% degli incassi. Per il fondatore questo significa che il modello funziona e che le due città principali vanno presidiate per prime, mentre zone come Ragusa e Agrigento (circa il 6% degli incassi ciascuna) richiedono una valutazione sulla convenienza di investimenti commerciali dedicati.

2. Le ore di punta sono il punto debole del servizio. Nelle quattro ore di maggiore domanda (9–11 e 17–19) la quota di richieste annullate sale al 12,2%, contro il 5,2% delle altre ore, e il tempo medio di erogazione passa da 131 a 138 minuti. La mappa di calore mostra con evidenza queste fasce. Ogni richiesta annullata è un incasso perso e un cliente potenzialmente insoddisfatto: l'indicazione

operativa è aumentare la disponibilità di operatori in quelle fasce, ad esempio con turni dedicati o incentivi ai partner.

3. La qualità del servizio varia molto tra le categorie. Le pulizie, seconda categoria per numero di richieste, hanno la quota più alta di interventi in ritardo (14,7%, contro il 3,1% della riparazione di elettrodomestici), mentre la climatizzazione concentra il 55% delle sue richieste nei mesi di giugno, luglio e agosto. Il tasso di ritorno, pari al 49,3%, indica una buona fidelizzazione: quasi un cliente attivo su due ha usato il servizio almeno due volte. Ridurre i ritardi nelle pulizie e pianificare in anticipo la capacità dei tecnici della climatizzazione per l'estate sono le due azioni con l'impatto più diretto sulla soddisfazione dei clienti.

Campi di applicazione
(Descrivere gli ambiti di applicazione dell'elaborato progettuale e i vantaggi derivanti della sua applicazione)**:**

Il prototipo è stato costruito per una startup di servizi a domicilio, ma la sua struttura è generale: qualunque attività che riceve richieste, le assegna a qualcuno che le esegue e le chiude con un importo e un tempo può riutilizzare lo stesso modello dei dati, cambiando solo i nomi delle categorie e le regole di generazione.

Ambiti di applicazione

- **Piattaforme di servizi on-demand**: servizi per la casa, consegne di cibo e spesa, trasporto con conducente, assistenza a domicilio, noleggio di mezzi. La tabella dei fatti resta la "richiesta", mentre le dimensioni si adattano (il fornitore può diventare il ristorante, l'operatore il rider).
- **Assistenza tecnica e customer service**: un'azienda che gestisce interventi di manutenzione o ticket di assistenza può usare lo stesso schema per misurare tempi di risoluzione, interventi fuori dai tempi contrattuali e carico per area.
- **Piccole e medie imprese di servizi**: molte PMI hanno i dati in fogli di calcolo separati e non dispongono di un reparto informatico. Una soluzione leggera basata su SQLite e Streamlit, gratuita e installabile su un normale computer, è un primo passo concreto verso decisioni basate sui dati.
- **Strutture ricettive e affitti brevi**: la stessa logica si applica alla gestione di prenotazioni, servizi di pulizia e manutenzione tra un soggiorno e l'altro, con la stagionalità come elemento centrale dell'analisi.
- **Didattica e formazione**: il progetto, completo di dati sintetici, test e documentazione, può essere usato come esempio introduttivo di data warehouse, ETL e business intelligence, perché si installa e si comprende in pochi minuti.

Vantaggi derivanti dall'applicazione

- **Una sola fonte di verità**: i dati dei diversi sistemi vengono riconciliati in un unico archivio, con codifiche uniformi; tutti in azienda guardano gli stessi numeri, calcolati con le stesse definizioni.
- **Decisioni più rapide**: il fondatore vede in pochi secondi l'andamento del business e può isolare una zona, una categoria o un periodo senza chiedere estrazioni a un tecnico.
- **Individuazione tempestiva dei problemi**: indicatori come la quota di annullate o di ritardi,

confrontati con il periodo precedente, segnalano un peggioramento prima che diventi visibile nei ricavi.

- **Qualità dei dati sotto controllo**: il report dell'ETL rende visibili gli errori presenti nei sistemi di origine, che possono così essere corretti alla fonte.
- **Costi contenuti**: tutti gli strumenti sono gratuiti e open source; il warehouse è un singolo file e non richiede server.
- **Pianificazione delle risorse**: le analisi per fascia oraria e per stagione aiutano a dimensionare il numero di operatori e a negoziare con i partner la disponibilità nei momenti di maggiore domanda.
- **Base per sviluppi successivi**: lo schema a stella è il punto di partenza naturale per analisi più avanzate, come previsioni della domanda o segmentazione dei clienti.

Valutazione dei risultati
(Descrivere le potenzialità e i limiti ai quali i risultati dell'elaborato sono potenzialmente esposti)

Potenzialità

Il prototipo raggiunge tutti gli obiettivi della traccia ed è pienamente funzionante: si installa ed esegue con pochi comandi, produce sempre gli stessi risultati a parità di parametri ed è coperto da test automatici. I suoi punti di forza sono la chiarezza del modello dei dati, che si spiega con un solo disegno; la tracciabilità delle trasformazioni, perché nessun dato viene scartato senza lasciare traccia; la coerenza degli indicatori, garantita da definizioni scritte e verificate da un test; la leggibilità della dashboard, che porta in primo piano i numeri più importanti e consente di filtrarli per periodo, zona e categoria.

La struttura a passi indipendenti rende il prototipo una buona base per un sistema reale: il generatore può essere sostituito da estrazioni dai sistemi aziendali senza modificare il warehouse e la dashboard, e lo schema a stella può essere esteso con nuove dimensioni e misure senza riprogettare quanto già fatto.

Limiti

- **Dati sintetici.** È il limite principale e una scelta imposta dalla traccia. Gli andamenti osservati riflettono le ipotesi del generatore (crescita del 6% al mese, picchi orari, stagionalità): la dashboard li rende visibili correttamente, ma non costituiscono una conoscenza sul mercato reale. Inoltre i dati reali presentano difetti più vari e meno prevedibili di quelli simulati.
- **Volumi ridotti.** Con 600 richieste in un anno molti valori, soprattutto se filtrati per zona e categoria, si basano su poche decine di casi e sono soggetti a forti oscillazioni casuali. Durante lo sviluppo ho verificato che, cambiando il seme, la serie mensile può variare in modo marcato.
- **Nessuna gestione dello storico nelle dimensioni.** Se un fornitore cambia categoria o un cliente cambia zona, il caricamento attuale sovrascrive il dato: le richieste passate verrebbero attribuite alla nuova zona, alterando le analisi storiche.
- **Caricamento completo a ogni esecuzione.** L'ETL ricrea il warehouse da zero. Con piccoli volumi è la soluzione più semplice e robusta, ma non è adatta a dati che crescono ogni giorno.
- **Controlli di qualità codificati.** Le regole di pulizia sono scritte nel codice e coprono i difetti noti; un difetto nuovo potrebbe passare inosservato o causare scarti non previsti.
- **Uso locale.** La dashboard funziona su un singolo computer, senza autenticazione e senza

aggiornamento automatico.

- **Tempo di erogazione semplificato.** È misurato come un unico intervallo tra richiesta e chiusura; in un caso reale sarebbe utile distinguere il tempo di attesa dell'operatore dal tempo dell'intervento.

Possibili miglioramenti

- **Utilizzo di dati reali.** Collegare l'ETL alle esportazioni dei sistemi aziendali o direttamente ai loro database. Prima di farlo, occorrerebbe una valutazione di impatto sulla protezione dei dati e la pseudonimizzazione dei codici dei clienti, nel rispetto del GDPR.
- **Storico delle modifiche nelle dimensioni.** Applicare la tecnica delle Slowly Changing Dimensions di tipo 2: quando un attributo cambia, si aggiunge una nuova versione della riga con date di validità. Le chiavi surrogate già presenti nello schema rendono questa estensione naturale.
- **Aggiornamento progressivo dei dati.** Passare a un caricamento incrementale che legge solo le richieste nuove o modificate dall'ultima esecuzione, e pianificarlo in automatico, ad esempio ogni notte.
- **Controlli automatici sulla qualità dei dati.** Formalizzare le regole in un catalogo di controlli dichiarativi (valori ammessi, intervalli, unicità, completezza), con soglie di allerta: ad esempio un avviso se la quota di righe scartate supera l'1%.
- **Scalabilità e condivisione.** Spostare il warehouse su un database server o in cloud e pubblicare la dashboard con accesso protetto, così che più persone possano consultarla.
- **Analisi più avanzate.** Analisi di coorte per misurare la fidelizzazione dei clienti nel tempo, previsione della domanda per zona e fascia oraria per pianificare gli operatori, indicatori di soddisfazione raccolti dall'app.

Considerazioni finali

Il project work mi ha permesso di affrontare un problema tipico della digitalizzazione d'impresa, la frammentazione dei dati, ripercorrendone l'intero ciclo: dalla simulazione delle fonti alla progettazione del modello, dalla pulizia dei dati alla presentazione dei risultati. Il risultato è un prototipo semplice ma completo, che dimostra come anche una piccola impresa possa dotarsi con strumenti gratuiti di un sistema di analisi affidabile, a patto di curare la chiarezza del modello, la correttezza delle trasformazioni e la definizione rigorosa degli indicatori.